

The DesiredGainR Breeder's Guide

Turning breeder intent into a defensible selection decision — Edition 1

Félicien Akohoue · PhD

01 August 2026

Contents

Executive summary	3
1 Purpose and scope	3
1.1 What the package does	3
1.2 What it deliberately does not do	3
1.3 Feature maturity	4
2 Before any recommendation	4
2.1 Minimum evidence	4
2.2 Covariance provenance and admissibility	5
2.3 Conditioning	5
2.4 Repairing an inadmissible matrix	5
2.5 Units and direction	5
3 Defining the objective	6
3.1 Why economic weights are difficult	6
3.2 The two directions of conversion	6
3.3 Is the objective attainable?	6
3.4 How much does the answer depend on the numbers?	7
3.5 What was the programme actually selecting for?	7
4 The index families and their variants	7
4.1 Weight-based families	7
4.1.1 <code>selection_index(method = "smith_hazel")</code>	7
4.1.2 <code>selection_index(method = "base")</code>	7
4.2 Desired-gain families	8
4.2.1 <code>selection_index(method = "pesek_baker")</code> and "yamada"	8

4.3	Rank and threshold families	8
4.3.1	<code>selection_index(method = "mulamba_mock")</code>	8
4.3.2	<code>selection_index(method = "independent_culling")</code>	8
4.3.3	<code>selection_index(method = "tandem")</code>	8
4.4	Constrained families	9
4.5	The iterative desired-gain index	9
4.5.1	<code>run_dgsi()</code>	9
4.6	The quadratic genomic index	10
4.6.1	<code>run_qgsi()</code>	10
5	Evaluating and comparing indices	10
5.1	The criteria	10
5.2	How each fails	10
5.3	Reusing a fitted index	11
6	Uncertainty	11
6.1	Why this section is not optional	11
6.2	<code>index_uncertainty()</code>	11
6.2.1	Choosing the degrees of freedom	11
6.2.2	Reading the result	12
6.3	<code>propagate_covariance_uncertainty()</code>	12
7	Multi-environment structure	12
8	End-to-end pipeline walkthrough	13
8.1	Phase 1 — Evidence	13
8.2	Phase 2 — Objective	13
8.3	Phase 3 — Index and ranking	13
8.4	Phase 4 — Uncertainty and sign-off	14
9	Interpreting diagnostics	14
9.1	Conditioning and stability	14
9.2	Selection and shortfall	14
10	Decision sign-off	14

11 Complete function map	15
11.1 Objective definition	15
11.2 Covariance handling	15
11.3 Index construction	15
11.4 Evaluation and reuse	15
11.5 Uncertainty	16
11.6 Multi-environment	16
11.7 Simulation	16
11.8 Reference values	16
References	16

Executive summary

DesiredGainR converts a breeder’s stated intent into index coefficients, a candidate ranking, and the diagnostics needed to defend the resulting decision. Its distinctive premise is that most programmes cannot supply economic weights but can state what they want to change, and that the two are convertible.

Four claims govern how the package should be read.

An objective is a hypothesis about attainability, not a target. Response is bounded by selection intensity and by available genetic variation. A stated set of desired gains is frequently impossible at any intensity the programme can run, and that is checkable before an index is built.

Every reported gain conditions on covariance matrices that are estimates. Multi-trait restricted maximum likelihood usually estimates more variance components than the trial has independent families to determine. Results that do not report how far they move under that uncertainty are incomplete.

Constrained objectives usually have closed-form answers. Where a restriction on the response is linear, searching for it is unnecessary and introduces a dependence on the number of replicates run.

A decision must remain defensible after the fact. The package records the provenance of every covariance matrix, refuses inadmissible inputs rather than reporting impossible quantities, and reports repairs rather than performing them silently.

No R knowledge is required to read this guide. The companion vignette *Defining a breeding objective* covers the same ground with runnable code.

1 Purpose and scope

1.1 What the package does

DesiredGainR sits between a fitted multi-trait model and a selection decision. It takes covariance matrices and candidate values as given, and returns coefficients, a ranking, expected responses and diagnostics.

1.2 What it deliberately does not do

Task	Why not	Use instead
Design a mating plan	Ranking candidates and allocating crosses are different problems with different objectives	A dedicated mating-design tool; HapBlockR for haplotype-aware allocation
Control inbreeding	Optimal contribution selection is not implemented; the simulation reports coancestry but does not optimise against it	A relationship-management tool
Fit the mixed model	Estimating G and P is upstream work	ASReml-R, sommer, breedR, StageWise; import with <code>import_covariance()</code>
Simulate a breeding programme	The simulation layer is narrow by design: it compares desired-gain directions, nothing more	AlphaSimR directly

1.3 Feature maturity

Not every layer is equally established. Treating them alike invites a decision to be defended on the weakest one.

Maturity	Components
Stable	Objective definition and feasibility; covariance diagnostics, bending and import
Beta	Classical index families; restricted and proportional-gain indices; the desired-gain index; the quadratic genomic index
Experimental	Covariance uncertainty propagation
Experimental, not for operational recommendation	Recurrent simulation; direction optimisation; diversity optimisation; clonal mode

2 Before any recommendation

2.1 Minimum evidence

An index cannot be built responsibly without all of the following.

Requirement	Why it matters	Failure mode if absent
Candidate values on a stated scale	The index is a linear combination; scale determines what it weights	Coefficients that silently favour whichever trait carries the largest variance
A genetic covariance G	Determines what selection transmits	No expected response can be computed
A phenotypic covariance P	Determines what selection acts on	Accuracy and heritability undefined
Favourable direction per trait	Which way is better is not inferable from the data	An index that selects against the objective
Units per trait	Needed to interpret any gain	Gains that cannot be compared with programme targets

2.2 Covariance provenance and admissibility

$\mathbf{G}$ is a component of $\mathbf{P}$, so the residual $\mathbf{P} - \mathbf{G}$ is itself a covariance matrix and must be positive semidefinite. This is not a formality. Every consequence of violating it is a number that looks valid and is impossible: a heritability above one, an accuracy above one, a squared correlation above one, a negative mean squared prediction error.

Checking the diagonal is **not sufficient**. The condition $\mathbf{G}_{jj} \leq \mathbf{P}_{jj}$ for every trait is necessary but not sufficient, because a linear combination of traits can have genetic variance exceeding its phenotypic variance while no single trait does. The eigenvalues of the difference are the correct test, and the package applies it.

When the pair fails, the message names the offending combination and the largest single-trait ratio, so that a correlation-driven failure is distinguishable from a variance-driven one.

2.3 Conditioning

Report the condition number of $\mathbf{G}$ before trusting any index built on original trait scales.

$$\kappa(\mathbf{G}) = \frac{\lambda_{\max}}{\lambda_{\min}}$$

A large value means the matrix is close to singular and the coefficients are unstable: small changes in the estimate produce large changes in the index. Standardise the traits and look again. `matrix_diagnostics()` reports κ , the eigenvalues, the numerical rank and positive definiteness.

2.4 Repairing an inadmissible matrix

Multi-trait REML routinely returns a $\mathbf{G}$ that is not positive definite. `bend_covariance()` repairs it through `ASRgenomics::G.tuneup()`, which is a wrapper around Higham’s alternating-projections algorithm.

Two of that function’s fixed choices matter and are surfaced in the result:

Setting	Consequence
<code>keepDiag = FALSE</code>	Trait variances are not preserved, so the heritabilities implied by the bent matrix are not those supplied. Read <code>max_variance_change</code> .
<code>posd.tol = 0.01</code>	Eigenvalues floored at a hundredth of the largest, capping κ near 100. Appropriate for an $n \times n$ relationship matrix; aggressive for a $p \times p$ trait covariance, where κ of 10^3 to 10^4 is ordinary and genuine.

Bending is a repair of an estimate, not an estimate. Report the adjustment alongside any result derived from a bent matrix.

2.5 Units and direction

Every trait needs a declared favourable direction. Traits where smaller is better — disease severity, days to maturity, plant height where reduction is wanted — are named in `lower_is_better`, and the package reverses their sign internally so that larger always means better in the analysis space.

Desired gains and economic weights are then stated as **non-negative magnitudes** in that favourable-direction space. Supplying a negative magnitude is an error, not a way of expressing direction.

3 Defining the objective

3.1 Why economic weights are difficult

The classical index needs a price for every trait: how much a unit of grain yield is worth relative to a unit of plant height. In principle these are economic quantities. In practice almost no programme has them, and two people who agree entirely about the intent will write down different weights.

What programmes have is an intent: *lift yield without letting the cycle get longer; hold disease resistance where it is and push everything else*. An intent is not a weight vector.

3.2 The two directions of conversion

The package treats the intent as the input and the weights as derived.

Forward. Given economic weights $\mathbf{a}$, the Smith–Hazel coefficients are

$$\mathbf{b} = \mathbf{P}^{-1}\mathbf{G}\mathbf{a}$$

and the expected correlated response is $\Delta = i\mathbf{G}\mathbf{b}/\sqrt{\mathbf{b}^T\mathbf{P}\mathbf{b}}$.

Backward. Given desired gains $\mathbf{d}$, the weight vector for which those gains *are* the Smith–Hazel optimum is

$$\mathbf{w} = \mathbf{G}^{-1}\mathbf{P}\mathbf{G}^{-1}\mathbf{d}$$

and conversely the gains implied by weights $\mathbf{w}$ are $\mathbf{d} = \mathbf{G}\mathbf{P}^{-1}\mathbf{G}\mathbf{w}$.

`implied_economic_weights()` and `implied_desired_gains()` perform these conversions. Read the implied weights for **sign surprises**: a negative implied weight on a trait you asked to improve means the index obtains more of that trait by selecting against it directly, because of how strongly it correlates with something else in the objective. That may be correct, but it should be a decision rather than an accident.

3.3 Is the objective attainable?

Desired gains have a direction and a magnitude. The direction is almost always achievable; the magnitude usually is not.

The set of responses attainable at selection intensity i is the ellipsoid

$$\mathbf{R}^T\mathbf{G}^{-1}\mathbf{P}\mathbf{G}^{-1}\mathbf{R} = i^2,$$

so the intensity required to attain a stated $\mathbf{d}$ is

$$i_{\text{required}} = \sqrt{\mathbf{d}^T\mathbf{G}^{-1}\mathbf{P}\mathbf{G}^{-1}\mathbf{d}}.$$

`gain_feasibility()` returns this and the selected fraction it implies. An intensity of 4.2 corresponds to selecting roughly one candidate in ten thousand: that is not a plan.

The remedy is to scale the whole vector down until the required intensity matches what the programme runs. The direction is preserved; only the ambition changes. This is the single most useful thing the package does before any index is fitted, and it costs nothing.

3.4 How much does the answer depend on the numbers?

`weight_sensitivity()` perturbs the objective and reports how far the selected set moves.

Outcome	Reading
Selected set stable under a 20% weight change	The recommendation is defensible in a meeting
Selected set moves substantially	The data cannot distinguish your objective from a neighbouring one; decide on other grounds

3.5 What was the programme actually selecting for?

`retrospective_weights()` recovers the weight vector implied by a decision already made, from the observed selection differential $\mathbf{s}$:

$$\mathbf{b} = \mathbf{P}^{-1}\mathbf{s}$$

Comparing the recovered objective against the stated one is often the only way to discover what a programme's objective has actually been.

4 The index families and their variants

Distinguish the biological question from the optimisation method. A more elaborate method is not automatically more appropriate. Use the simplest method that represents the programme's actual objective and constraints.

4.1 Weight-based families

4.1.1 `selection_index(method = "smith_hazel")`

Question: *given prices for every trait, what maximises net merit?*

$$\mathbf{b} = \mathbf{P}^{-1}\mathbf{G}\mathbf{a}$$

The optimum index when $\mathbf{a}$ is genuinely known. Requires $\mathbf{G}$, $\mathbf{P}$ and economic weights.

4.1.2 `selection_index(method = "base")`

Question: *what is the simplest defensible benchmark?*

$$\mathbf{b} = \mathbf{a}$$

The weights are applied directly to the trait values with no covariance adjustment. Requires no $\mathbf{P}$. Always worth computing: if the optimum index barely beats the base index, the covariance structure is not doing much work and the extra estimation risk may not be worth carrying.

4.2 Desired-gain families

4.2.1 `selection_index(method = "pesek_baker")` and `"yamada"`

Question: *given a direction I want to move in, what index delivers it?*

$$\mathbf{b} = \mathbf{G}^{-1}\mathbf{d} \quad \text{and} \quad \mathbf{b} = \mathbf{P}^{-1}\mathbf{G}(\mathbf{G}\mathbf{P}^{-1}\mathbf{G})^{-1}\mathbf{d}$$

These are the same index. For invertible $\mathbf{G}$,

$$\mathbf{P}^{-1}\mathbf{G}(\mathbf{G}\mathbf{P}^{-1}\mathbf{G})^{-1}\mathbf{d} = \mathbf{P}^{-1}\mathbf{G} \mathbf{G}^{-1}\mathbf{P}\mathbf{G}^{-1}\mathbf{d} = \mathbf{G}^{-1}\mathbf{d}.$$

They are not alternatives to be compared, and a difference between them indicates numerical instability rather than a substantive choice.

The defining property is that the expected response is **exactly proportional to $\mathbf{d}$** :

$$\Delta = i \frac{\mathbf{G}\mathbf{b}}{\sqrt{\mathbf{b}^T\mathbf{P}\mathbf{b}}} = \frac{i}{\sqrt{\mathbf{b}^T\mathbf{P}\mathbf{b}}} \mathbf{d}.$$

This is what makes the family useful and what makes its estimation error measurable; see the chapter on uncertainty below.

4.3 Rank and threshold families

4.3.1 `selection_index(method = "mulamba_mock")`

Question: *the traits are on incomparable scales and I distrust the covariances — can I still combine them?*

Candidates are ranked within each trait and the ranks summed, optionally weighted. Uses no covariance matrix at all, so it is immune to a poorly estimated $\mathbf{G}$ and cannot report an expected genetic response.

4.3.2 `selection_index(method = "independent_culling")`

Question: *some traits are pass or fail, not tradeable.*

A candidate failing **any** threshold is never selected. `n_select` is a maximum, not a quota: if seven candidates pass and twenty were requested, seven are returned and the shortfall is reported with the gates each failure violated.

This is deliberate. Filling the shortfall would mean trading a failed trait against a good one, which is what an index does — so if that is what is wanted, use an index.

4.3.3 `selection_index(method = "tandem")`

Question: *only one trait matters per stage.*

Sequential selection on one trait at a time in a stated order. Historically common, rarely optimal, retained as a comparator.

4.4 Constrained families

`restricted_index()` implements the closed-form constrained indices. Where a restriction on the response is linear, these give exactly what an iterative search approaches, without the replicate dependence.

Let $\mathbf{C}$ be the $k \times p$ matrix selecting the constrained traits. Expected response is proportional to $\mathbf{G}\mathbf{b}$, so a zero-response restriction is $\mathbf{C}\mathbf{G}\mathbf{b} = \mathbf{0}$, and

$$\mathbf{b} = [\mathbf{I} - \mathbf{P}^{-1}\mathbf{G}\mathbf{C}^{\top}(\mathbf{C}\mathbf{G}\mathbf{P}^{-1}\mathbf{G}\mathbf{C}^{\top})^{-1}\mathbf{C}\mathbf{G}] \mathbf{P}^{-1}\mathbf{G}\mathbf{a}.$$

Method	Question	Constraint
<code>kempthorne_nordskog</code>	Hold these traits still	Zero expected response in the restricted traits
<code>restricted_smith_hazel</code>	As above, projection form	Algebraically identical; retained because most texts present this form
<code>tallis</code>	Move these traits in this ratio	Responses held in the supplied proportions; magnitude free
<code>mallard</code>	Move these traits by these amounts	Responses held at specified values; magnitude fixed
<code>harville</code>	Weight the constraint against merit	Restriction applied as a penalty; penalty = Inf recovers Kempthorne–Nordskog

A restriction can never raise the correlation with net merit. The question is whether the loss is acceptable, and that is a breeding decision rather than a statistical one. Compare R_{HI} between the restricted and unrestricted indices to price it.

4.5 The iterative desired-gain index

4.5.1 `run_dgsi()`

Searches stochastically for the desired-gain vector whose realised response best matches the target, then reports the resulting coefficients.

Variant	Argument	Use
Selection rule	<code>select_mode = "top_n"</code>	Rank on the index alone
Replicate choice	<code>select_mode = "eligible_top_n"</code>	Apply trait floors first, then rank
	<code>replicate_selection = "holdout"</code>	Default. Chooses the winning search on candidates held out of that comparison, so the choice does not depend on the candidates finally selected
Estimated $\mathbf{P}$	<code>replicate_selection = "training"</code>	Earlier behaviour; a minimum over n_rep draws and biased downward
	<code>allow_incompatible_estimated_P</code>	Continue when a $\mathbf{P}$ estimated from reference candidates falls below $\mathbf{G}$ by more than sampling error explains. Recorded as an override

`run_dgsi()` reports two different quantities that must not be confused:

Quantity	Meaning
<code>realised_response</code>	The standardised differential between the selected candidates and the whole set. A property of who was picked
<code>theoretical_response</code>	$i \mathbf{Gb} / \sqrt{\mathbf{b}^\top \mathbf{Pb}}$ — what the next generation inherits

A large differential with a small transmitted response means selection is acting mostly on non-heritable variation.

4.6 The quadratic genomic index

4.6.1 `run_qgsi()`

For genomic estimated breeding values with both linear and quadratic economic weights,

$$I = \mathbf{w}^\top \boldsymbol{\gamma} + \boldsymbol{\gamma}^\top \mathbf{W} \boldsymbol{\gamma},$$

with

$$\mathbb{E}(I) = \text{tr}(\mathbf{W}\boldsymbol{\Gamma}), \quad \text{Var}(I) = \mathbf{w}^\top \boldsymbol{\Gamma} \mathbf{w} + 2 \text{tr}(\mathbf{W}\boldsymbol{\Gamma}\mathbf{W}\boldsymbol{\Gamma}).$$

Accuracy and mean squared prediction error are reported only when `true_G` is supplied, which is possible in simulation. The package then requires $\mathbf{G}_{\text{true}} - \boldsymbol{\Gamma}$ to be positive semidefinite, since a prediction cannot vary more than what it predicts.

5 Evaluating and comparing indices

5.1 The criteria

Criterion	Definition	Reading
R_{HI}	$\frac{\mathbf{b}^\top \mathbf{Ga}}{\sqrt{\mathbf{b}^\top \mathbf{Pb}} \sqrt{\mathbf{a}^\top \mathbf{Ga}}}$	Correlation between the index and net merit. Maximising it maximises response in aggregate merit
ΔH	$i R_{HI} \sqrt{\mathbf{a}^\top \mathbf{Ga}}$	Expected gain in aggregate merit
Δ_j	$i (\mathbf{Gb})_j / \sqrt{\mathbf{b}^\top \mathbf{Pb}}$	Expected response for trait j
RE	$\Delta_m / (i \mathbf{G}_{mm} / \sqrt{\mathbf{P}_{mm}})$	Efficiency relative to direct selection on the main trait
h_I^2	$\mathbf{b}^\top \mathbf{Gb} / \mathbf{b}^\top \mathbf{Pb}$	Heritability of the index treated as a trait
CV_I	$100 s_I / \bar{I} $	Coefficient of variation of the index scores

5.2 How each fails

Relative efficiency below 1 is not a defect. A multi-trait index deliberately gives up response in any single trait to gain elsewhere. Read it alongside what direct selection would have sacrificed.

R_{HI} **requires a definition of net merit.** Desired gains do not provide one. Unless one common `aggregate_weights` vector is supplied, `DesiredGainR` returns NA for R_{HI} and ΔH rather than changing the definition of merit from one desired-gain direction to another. The implied weights $\mathbf{w} = \mathbf{G}^{-1}\mathbf{P}\mathbf{G}^{-1}\mathbf{d}$ remain available as an explicit diagnostic through `implied_economic_weights()`; they are not a common objective for cross-method comparison.

CV_I **is undefined when the index is centred.** Standardising or centring the traits puts the index mean at zero and the ratio diverges. The package withholds it and says so rather than reporting an enormous number.

h_I^2 **is the only accuracy available before economic weights are supplied,** which makes it the right criterion for comparing desired-gain indices with each other.

5.3 Reusing a fitted index

`predict()` applies a fitted index to a new candidate set, reusing the centring and scaling constants **from fitting** rather than recomputing them. That is deliberate: recomputing would score each cycle on its own mean, so a set that had advanced genetically would appear identical to one that had not.

The consequence is that new data must be on the same scale and in the same units. Where a trial has been re-standardised, refit rather than predict.

6 Uncertainty

6.1 Why this section is not optional

Every criterion above treats $\mathbf{G}$ and $\mathbf{P}$ as known. They are estimates, usually from fewer independent families than the covariance matrix has free parameters, and the coefficients are a non-linear function of them.

An index whose coefficients cannot be distinguished from zero is not a different index from one whose coefficients are sharply determined — yet the point estimate presents them identically.

6.2 `index_uncertainty()`

Resamples the covariance matrices and refits.

$$\mathbf{G}^* \sim \mathcal{W}_p(\nu_g, \mathbf{G}/\nu_g), \quad \mathbf{E}^* \sim \mathcal{W}_p(\nu_e, \mathbf{E}/\nu_e), \quad \mathbf{P}^* = \mathbf{G}^* + \mathbf{E}^*$$

The **genetic and residual** matrices are drawn, not the genetic and phenotypic ones, because $\mathbf{G}$ and $\mathbf{P}$ are built from the same records and drawing them independently would produce pairs that could not both be true.

6.2.1 Choosing the degrees of freedom

This is the input that governs every interval width, and the one users get wrong.

ν_g counts **independent genetic units**, not plots and not observations.

Design	ν_g is on the order of
Half-sib trial	Number of families
Diallel	Number of parents

Design	ν_g is on the order of
Clonally replicated trial	Number of distinct clones

Replication within a genotype sharpens the residual matrix, not the genetic one. Supplying the plot count where the family count was meant understates every interval by roughly the square root of the replication.

6.2.2 Reading the result

For the desired-gain families the reported cost has an **exact reference point**. Since $\mathbf{G}\mathbf{b} = \mathbf{d}$ identically, an index fitted on the true $\mathbf{G}$ delivers the requested gains in the requested proportions. The cosine between $\mathbf{G}^*\hat{\mathbf{b}}$ and $\mathbf{d}$ therefore isolates estimation error with nothing confounding it.

An alignment interval whose lower bound sits well below one means the desired gains were specified more precisely than the data can deliver them. The honest response is to state fewer of them.

6.3 `propagate_covariance_uncertainty()`

Extends the same idea to a multi-cycle recommendation. Directions are re-evaluated across draws from the sampling distribution of the covariance, **rebuilding the founder population for each draw**, because $\mathbf{G}$ is the architecture the traits are built from rather than a matrix the simulation multiplies by. Each sampled residual matrix is passed directly to AlphaSimR’s phenotype model, so both genetic and residual uncertainty affect the cycles.

The result separates the two sources of uncertainty:

Source	Falls with	Remedy
Monte Carlo simulation	More replicates	Compute
Covariance estimation	Nothing the simulation can do	More independent genetic units in the trial

When the covariance component dominates, running more replicates is wasted effort, and the result says so.

7 Multi-environment structure

A multi-environment index is a single-environment index on an expanded trait set. `expand_environments()` builds the separable covariance

$$\text{Cov}(g_{je}, g_{kf}) = \mathbf{G}_{jk} \mathbf{C}_{ef}$$

where $\mathbf{C}$ is the between-environment genetic correlation matrix.

Mean off-diagonal of $\mathbf{C}$	Reading
Near 1	Environments rank genotypes almost identically; a single-environment index on trait means gives nearly the same answer at a fraction of the complexity
Middling	Interaction is material and worth modelling
Near 0	Environments rank genotypes very differently; treating them as one target population may not be appropriate

Environment-specific economic weights are supported: yield in a marginal environment serving a small area contributes less to net merit than yield in the main target environment.

Setting `include_stability = TRUE` returns per-trait response contrasts measuring departure from a genotype's own cross-environment mean. Pass `expanded$stability$constraint_matrix` to `restricted_index()`. A genotype whose deviations are uniformly zero performs at its own mean everywhere, which is what stability means here. Constraining it costs response in the mean; use `restricted_index()` to price that trade rather than assume it.

The reported `interaction_share` is the heuristic one minus the mean environmental correlation. It is a screening summary, not a general interaction-variance identity.

8 End-to-end pipeline walkthrough

8.1 Phase 1 — Evidence

Stage	Action	Function	Gate
1	Assemble candidate values, units and directions	—	Every trait has a stated direction and unit
2	Import or estimate G and P	<code>import_covariance()</code> , <code>estimate_genetic_covariances()</code>	Trait order verified, not summed
3	Check conditioning	<code>matrix_diagnostics()</code>	κ recorded; standardise if large
4	Check admissibility	automatic	P – G positive semidefinite
5	Repair if required	<code>bend_covariance()</code>	Adjustment recorded and reported

8.2 Phase 2 — Objective

Stage	Action	Function	Gate
6	State desired gains or weights	—	Non-negative magnitudes in the favourable direction
7	Test attainability	<code>gain_feasibility()</code>	Required intensity within programme capability
8	Read the implied objective	<code>implied_economic_weights()</code>	No unexplained sign reversals
9	Test sensitivity	<code>weight_sensitivity()</code>	Selected set stable under plausible perturbation
10	Recover the historical objective	<code>retrospective_weights()</code>	Stated and revealed objectives compared

8.3 Phase 3 — Index and ranking

Stage	Action	Function	Gate
11	Fit the chosen family	<code>selection_index()</code> , <code>restricted_index()</code> , <code>run_dgsi()</code> , <code>run_qgsi()</code>	Method justified against the objective

Stage	Action	Function	Gate
12	Evaluate	<code>evaluate_index()</code> , <code>summary()</code>	Criteria within their natural bounds
13	Compare families	—	Overlap of selected sets reported, not only criteria
14	Verify constraints	<code>\$constraint</code>	Largest violation negligible

8.4 Phase 4 — Uncertainty and sign-off

Stage	Action	Function	Gate
15	Propagate covariance error	<code>index_uncertainty()</code>	ν_g counts genetic units, not plots
16	Look ahead, if relevant	<code>simulate_selection_cycles()</code> <code>optimize_desired_gains()</code>	Treated as research output
17	Record the decision	—	Provenance, repairs and overrides all documented

9 Interpreting diagnostics

9.1 Conditioning and stability

Symptom	Likely cause	Action
$\kappa(\mathbf{G}) > 10^4$ on raw scales	Trait scales differ by orders of magnitude	Standardise and refit
Coefficients change sign between families	Near-singular $\mathbf{G}$	Report conditioning; prefer the base index
Wide coefficient intervals	Few independent genetic units	State fewer desired gains
Alignment lower bound well below 1	Objective over-specified for the data	Reduce the number of constrained traits

9.2 Selection and shortfall

Symptom	Meaning
Fewer selected than requested under culling	Correct behaviour; the gates are hard
Large empirical differential, small transmitted response	Selection acting on non-heritable variation
Two indices with different coefficients selecting the same lines	The choice between them does not matter here

10 Decision sign-off

Before a selection decision is approved, each of the following should have a recorded answer. An unanswered item means the analysis is incomplete rather than wrong.

1. What candidate set and data release were analysed?

2. What objective was declared, in what units, and before or after seeing the ranking?
3. Was feasibility checked, and was the required intensity attainable?
4. Do the implied economic weights carry the expected signs?
5. Was the covariance pair admissible, and was anything bent? By how much?
6. What is the condition number, and were traits standardised?
7. Which index family was used, and why that one?
8. Does the selected set survive a sensitivity analysis?
9. Was covariance uncertainty propagated, and with what ν_g ?
10. Is the reported response transmitted genetic response or an observed differential?
11. Were any overrides used, such as `allow_incompatible_estimated_P`?

11 Complete function map

11.1 Objective definition

Function	Purpose
<code>implied_economic_weights()</code>	Weights implied by stated desired gains
<code>implied_desired_gains()</code>	Gains implied by stated weights
<code>gain_feasibility()</code>	Required intensity and selected fraction
<code>weight_sensitivity()</code>	Stability of the selected set under perturbation
<code>retrospective_weights()</code>	Objective implied by a past decision
<code>effective_weights()</code>	Weights an existing coefficient vector behaves as

11.2 Covariance handling

Function	Purpose
<code>estimate_genetic_covariance()</code>	Labelled approximations with recorded estimand
<code>import_covariance()</code>	Adapters for ASReml-R, sommer, breedR, BGLR, StageWise
<code>matrix_diagnostics()</code>	Eigenvalues, condition number, rank, definiteness
<code>bend_covariance()</code>	Repair to positive definiteness, with the adjustment reported

11.3 Index construction

Function	Purpose
<code>selection_index()</code>	Seven classical families through one interface
<code>restricted_index()</code>	Kempthorne–Nordskog, Tallis, Mallard, Harville, restricted Smith–Hazel
<code>run_dgsi()</code>	Iterative desired-gain index with replicate diagnostics
<code>run_qgsi()</code>	Quadratic genomic selection index
<code>run_dgsi_qgsi_pipeline()</code>	Both, with comparison
<code>compare_dg_and_qgsi()</code>	Rank agreement between the two

11.4 Evaluation and reuse

Function	Purpose
<code>evaluate_index()</code>	The standard criteria
<code>summary()</code> , <code>coef()</code>	Tidy tables of coefficients, response and criteria
<code>predict()</code>	Apply a fitted index to a new candidate set

11.5 Uncertainty

Function	Purpose
<code>index_uncertainty()</code>	Coefficient intervals, alignment, rank stability
<code>draw_covariance_pairs()</code>	Jointly admissible covariance draws
<code>propagate_covariance_uncertainty()</code>	Covariance error in a multi-cycle recommendation

11.6 Multi-environment

Function	Purpose
<code>expand_environments()</code>	Separable trait-by-environment covariance
<code>widen_environments()</code>	Reshape long-format trial data

11.7 Simulation

Function	Purpose
<code>founder_haplotypes()</code>	Phased marker data into AlphaSimR founders
<code>haplotypes_from_inbred_dosage()</code>	Dosage to haplotypes for inbred material
<code>dosage_diagnostics()</code>	Residual heterozygosity and missingness
<code>founder_population()</code>	Calibrate trait architecture to $\mathbf{G}$
<code>simulate_selection_cycles()</code>	Forward simulation over cycles
<code>optimize_desired_gains()</code>	Search over desired-gain directions

11.8 Reference values

Function	Purpose
<code>rahimi_debnath_2023()</code>	Frozen published values for external validation

References

- Smith, H.F. (1936) A discriminant function for plant selection. *Annals of Eugenics* **7**, 240–250.
- Hazel, L.N. (1943) The genetic basis for constructing selection indexes. *Genetics* **28**, 476–490.
- Kempthorne, O. and Nordskog, A.W. (1959) Restricted selection indices. *Biometrics* **15**, 10–19.
- Tallis, G.M. (1962) A selection index for optimum genotype. *Biometrics* **18**, 120–122.
- Pesek, J. and Baker, R.J. (1969) Desired improvement in relation to selection indices. *Canadian Journal of Plant Science* **49**, 803–804.

- Mallard, J. (1972) La théorie et le calcul des index de sélection avec restrictions. *Biometrics* **28**, 713–735.
- Kemp, C.D. and Harville, D.A. (1975) Index selection with proportionality constraints. *Biometrics* **31**, 223–225.
- Yamada, Y., Yokouchi, K. and Nishida, A. (1975) Selection index when genetic gains of individual traits are of primary concern. *Japanese Journal of Genetics* **50**, 33–41.
- Higham, N.J. (2002) Computing the nearest correlation matrix. *IMA Journal of Numerical Analysis* **22**, 329–343.
- VanRaden, P.M. (2008) Efficient methods to compute genomic predictions. *Journal of Dairy Science* **91**, 4414–4423.
- Gaynor, R.C., Gorjanc, G. and Hickey, J.M. (2021) AlphaSimR: an R package for breeding program simulations. *G3* **11**, jkaa017.
- Rahimi, M. and Debnath, S. (2023) Estimating optimum and base selection indices in plant and animal breeding programs by development new and simple SAS and R codes. *Scientific Reports* **13**, 18977.